GenAI

Generative AI

↓

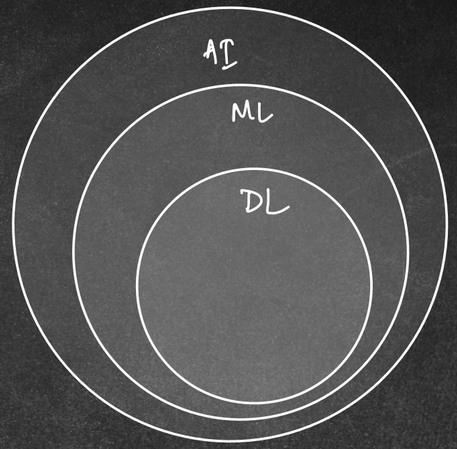
create new content

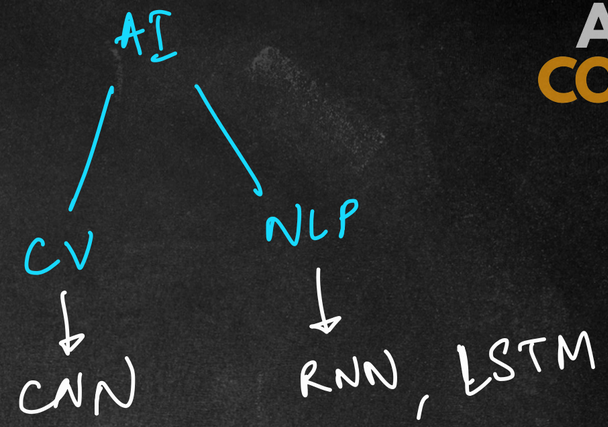
ANN

RNN → LSTM

CNN

Transformer





images/videos → dalle., midjourney

text → chatgpt, claude, gemini

audio/sound/music → sound draw

code → copilot

⋮

APNA COLLEGE

LLMs

Large Language Models

- ✓ Billions of parameters
- class of models in NLP
- ✓ trained on huge datasets
- ✓ capabilities ↑↑

How are you?

Q: why should I code in Python?

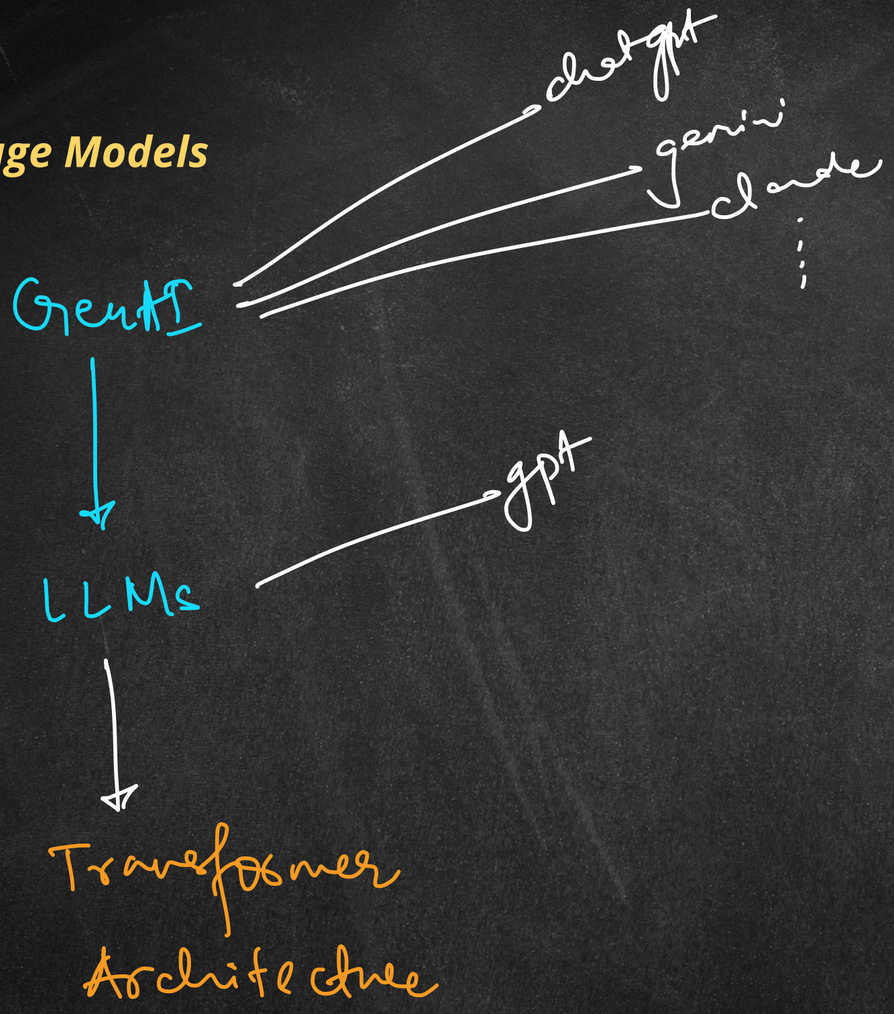
Ans: Python is a great language

GenAI → text / languages → LLMs

APNA COLLEGE

LLMs

Large Language Models



APNA COLLEGE

Transformers

NN arch.

2017 ⇒ "Attention is all you need"

Attention ⇒ parallelly

Sequence → text

"I like Python"

RNN, LSTM, CNN

step by step (sequentially)

sequence models

APNA COLLEGE

Transformers

Embedding → RNN

word embeddings → vectors

text → "I like Python" → numbs

I, like, Python

[0.1, 0.7] [0.32, 0.41] [0.8, 0.2]

features

Is it a language? proj

Is it an object?

GPT-3 → 12k+ → 12,288 dimensions in embeddings

APNA COLLEGE

Transformers

Static vs Contextual Embedding

I will read a book.

I will book movie tickets.

APNA COLLEGE

Word2Vec

↓

2 layer NN

↓

Static embedding

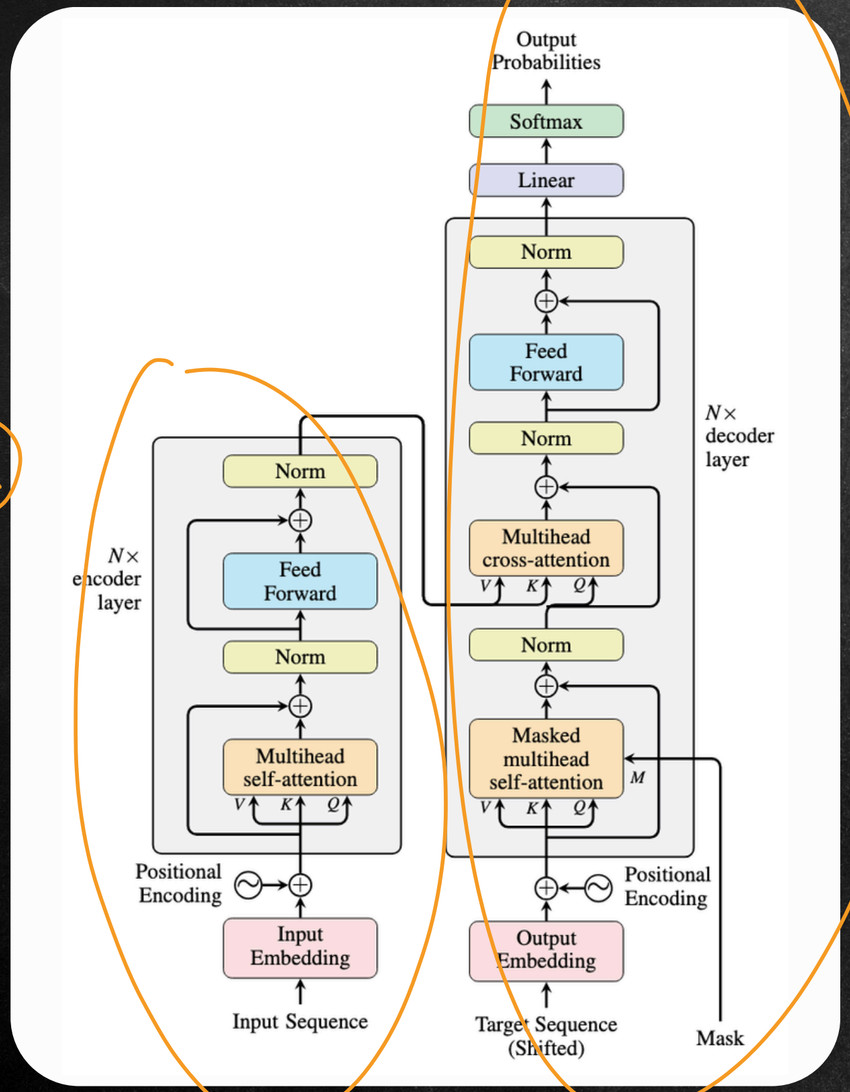
book → [0.32, 0.5, 0.01]

Python → snake

↳ prog. language

Transformers

Architecture



(1) encoder — understand
(2) decoder — generate

"I like Python" — inp sequence ⇒ contextual embeddings

I
liked
Python

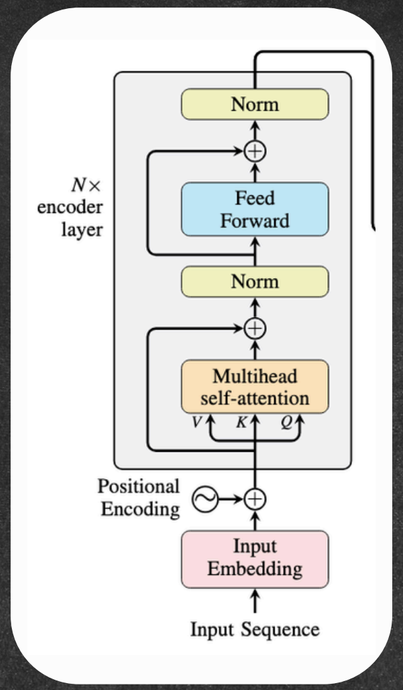
(1) summarize
(2) sentiment analysis
(3) text classification
(4) speech translation

① Tokenization

② Contextual embeddings

③ pass to the decoder

"I like Python"



Encoder

"I like Python"

[I, like, Python]

0.1	0.5	0.9
0.3	0.8	0.2
....	....	....
0.0	0.2	0.3

Static embedding ⟶ contextual

+

Vocab

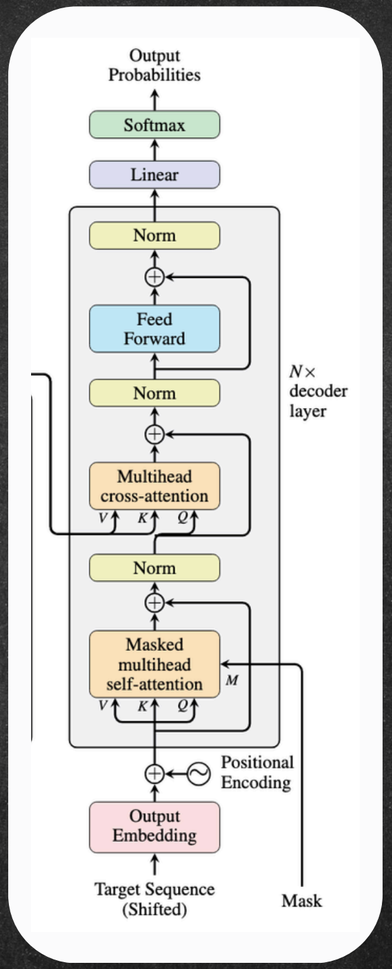
APNA COLLEGE

"I like Python"

[I, like, Python]

0.1	0.5	0.9
0.3	0.8	0.2
....	....	....
0.0	0.2	0.3

generate o/p one at a time



Decoder

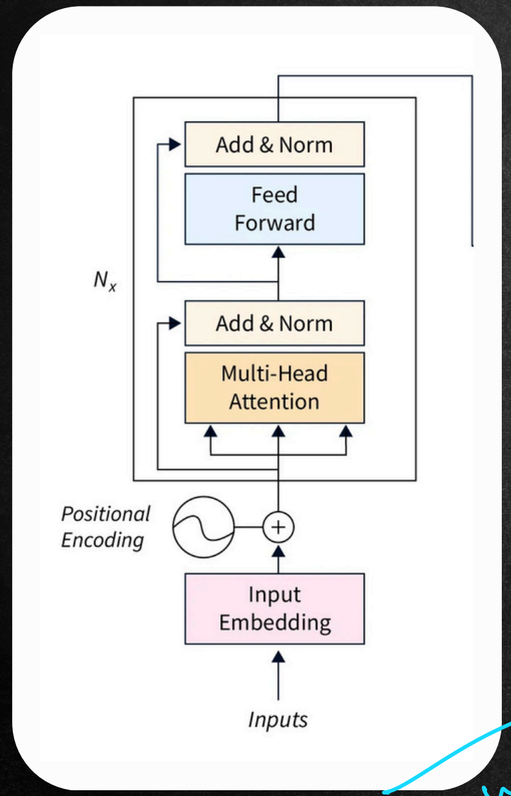
<START>
मुझे
पाइथन
पसंद
है
<EOS> end of sequence

मुझे → 70%
मैं → 10%
वह → 0.2%
यहाँ → 0.05%

APNA COLLEGE

Different Tranformer Architectures

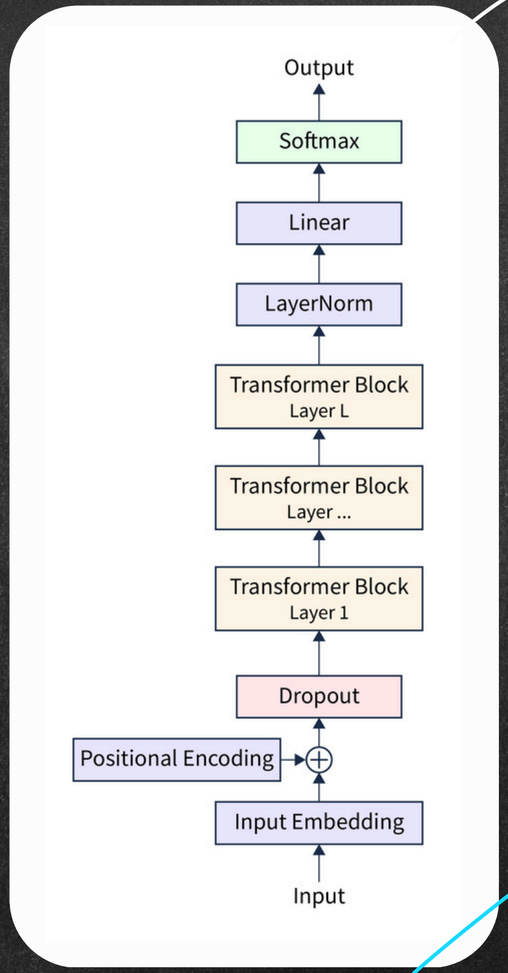
- text classification
- Sentiment analysis



understanding

BERT

encoder - only transformer arch.



text generation

GPT / Claude

decoder - only transformer arch.

① conversation

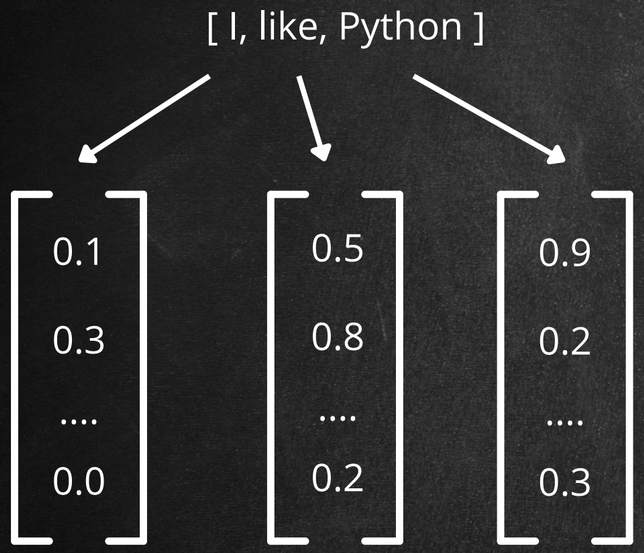
② easier to train

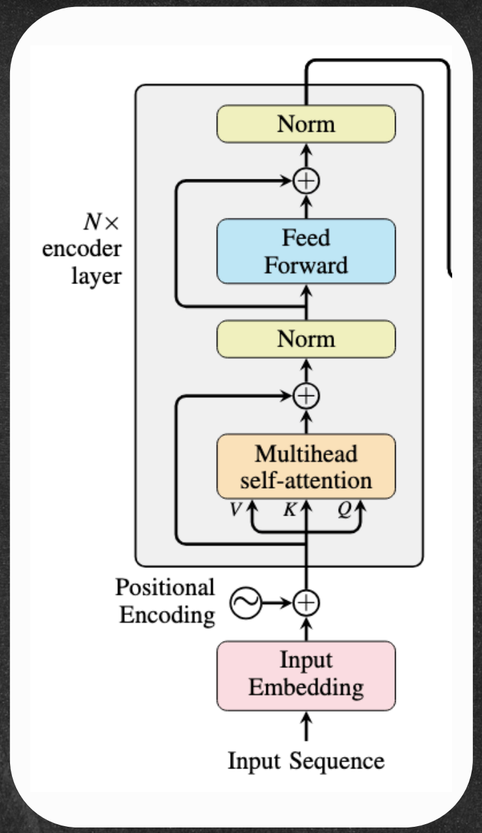
token by token

APNA COLLEGE

Encoder

Positional Encoding





I like Python

Python like I

RNN → sequentially

I, like, Python

Transformers

↓

Parallelization

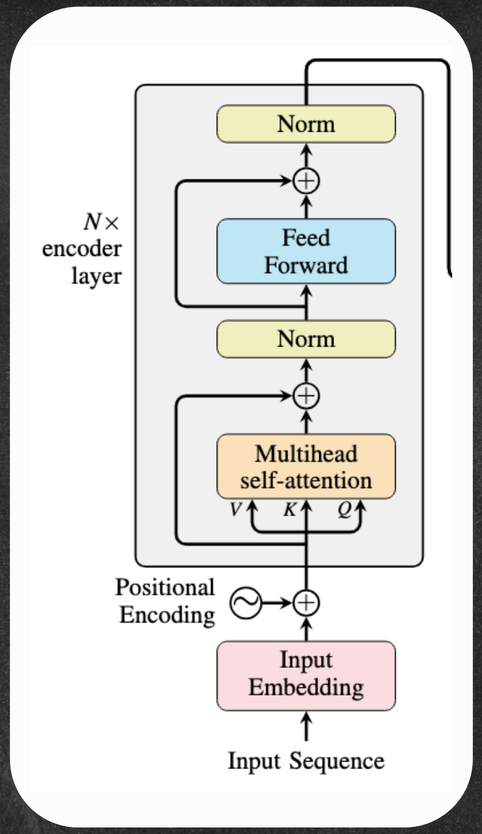
— training faster

— performance ↑

APNA COLLEGE

Encoder

Positional Encoding

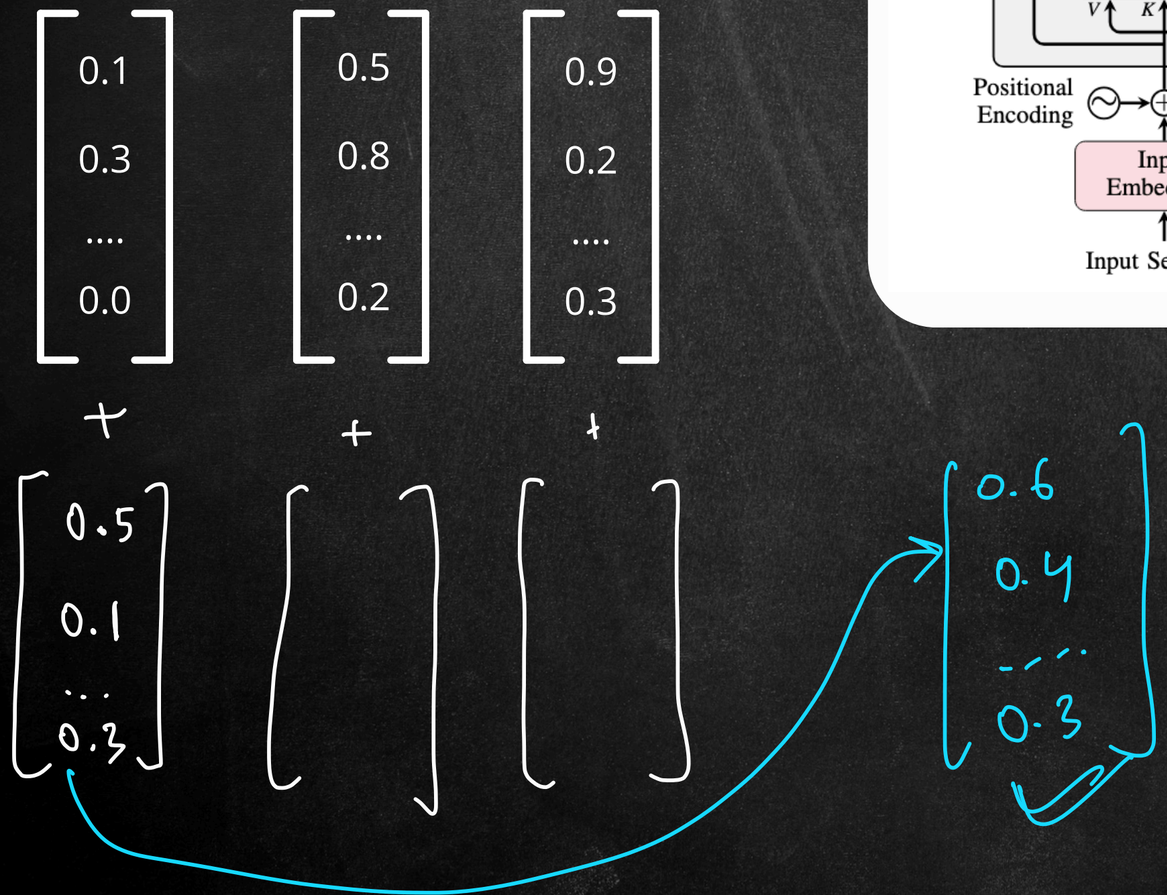


final o/p = I/p embedding + Position embeddings

$$PE(pos, 2i) = \sin\left(pos / 10{,}000^{2i/d_{model}}\right)$$

$$PE(pos, 2i+1) = \cos\left(pos / 10{,}000^{2i/d_{model}}\right)$$

[I, like, Python]

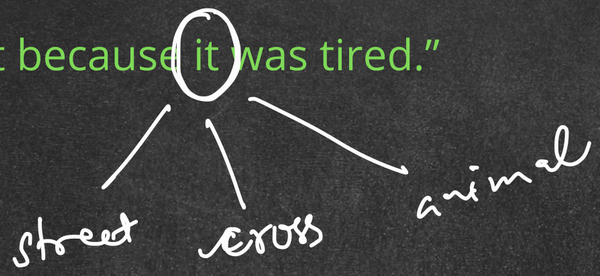


Attention ⇒ focus

what/who should it pay attention to?

Attention is all you need.

"The animal didn't cross the street because it was tired."

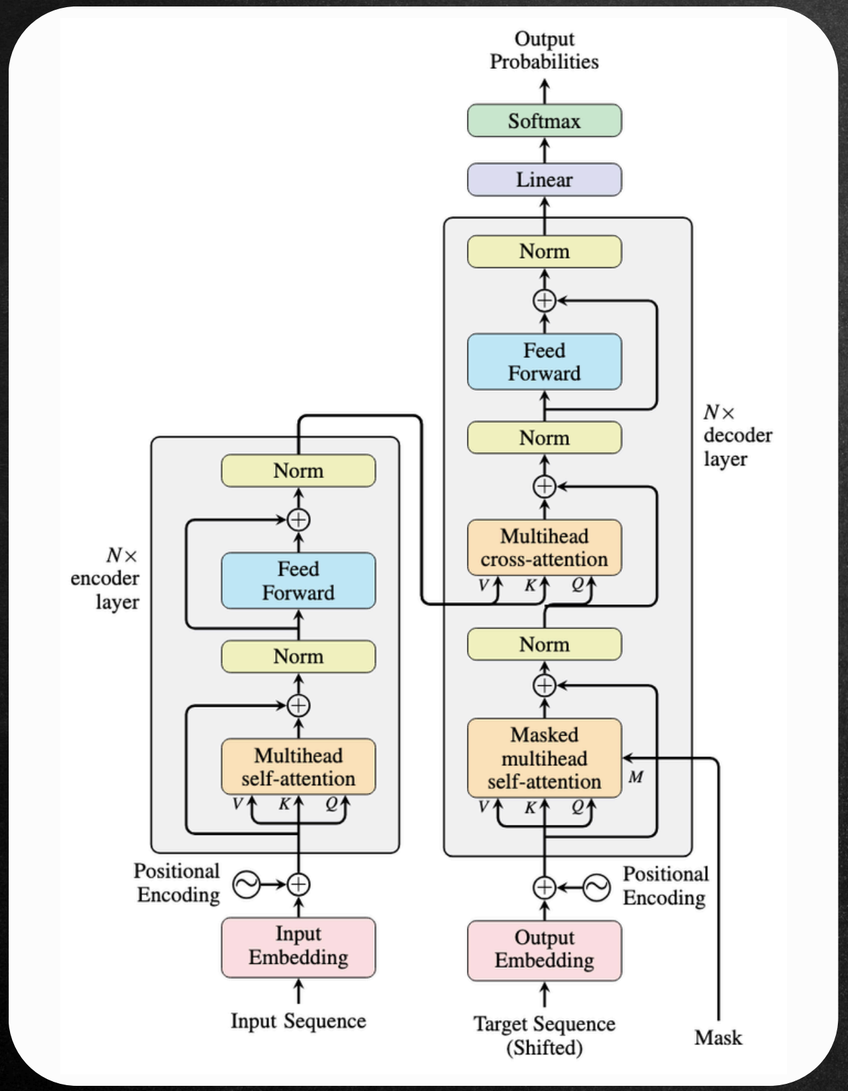


mapping

Q → K, V

Attention

Attention is all you need.



Attention

Attention is all you need.

Teacher → Transformer

Query (Q): what is the capital of India?

R: geography (Key) → V: capital of India

S: math (Key) → V: 2+2=4

G: history (Key) → V: Independence/Constitution of India

strong match = 0.85

low = 0.05

med. = 0.1

3 matrices

Q, K, V

attention score ⇒ scaled dot product

Attention (Q, K, V)

APNA COLLEGE

Attention

Attention is all you need.

input seq → n token

d → dimension of model

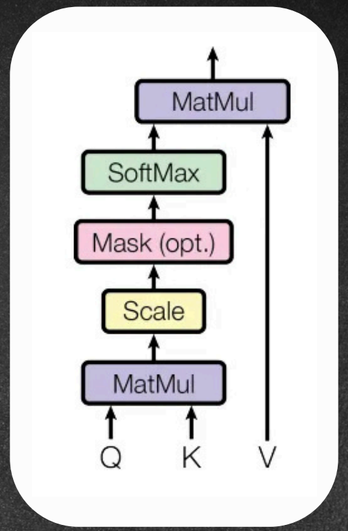
① $X = \begin{bmatrix} 0.3 \\ 0.4 \\ \cdots \\ 0.1 \end{bmatrix} \Rightarrow n \times d$

$Q = X \cdot W_Q \rightarrow d \times d_k$

$K = X \cdot W_K \rightarrow d \times d_k$

$V = X \cdot W_V \rightarrow d \times d_v$

↳ weight matrices



Scaled Dot Product Attention

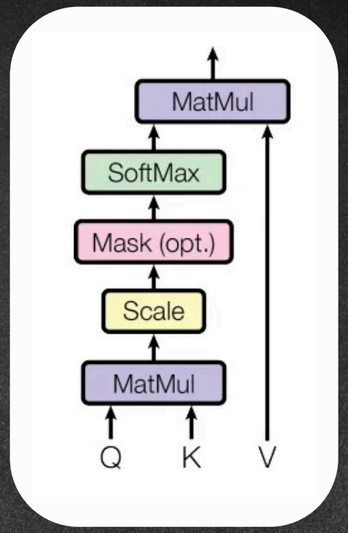
⇓

Attention(Q, K, V)

APNA COLLEGE

Attention

Attention is all you need.



geography — 0.05
maths → 0.95

APNA COLLEGE

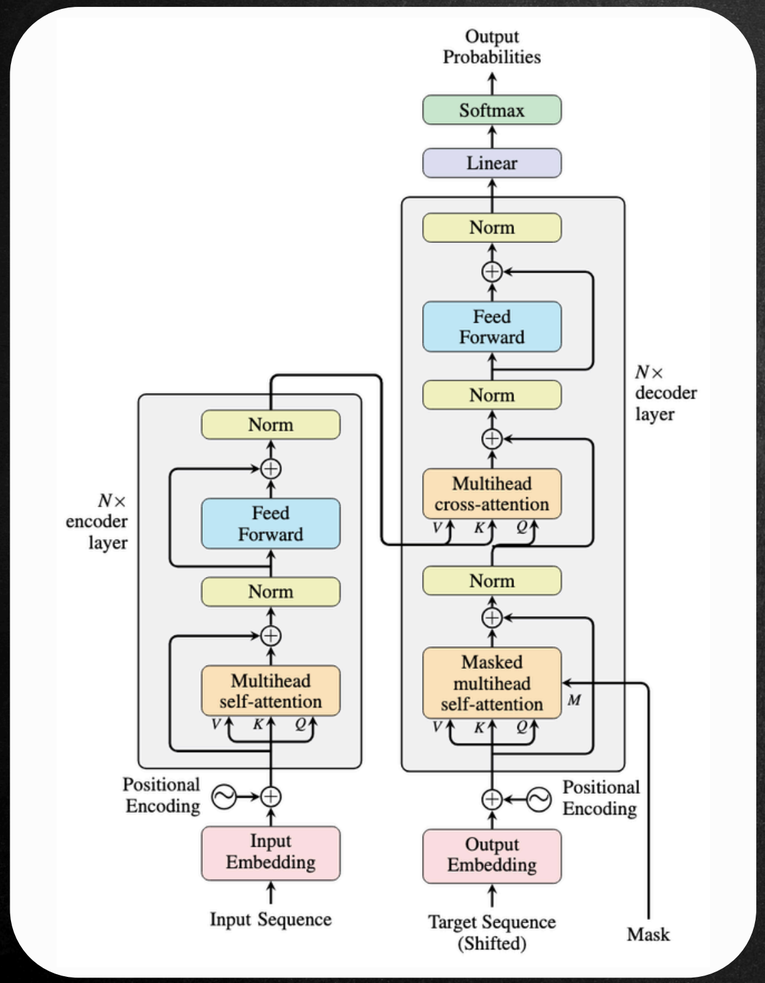
Q, K, V

(dot product)

1. similarity score ⇒ $Q \cdot K^T$
2. scaling ⇒ $\frac{Q \cdot K^T}{\sqrt{d_k}}$
3. softmax ⇒ $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$ → probabilities
4. $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$

Attention

Self-Attention



(SDP) scaled dot product → self-Attention → contextual embeddings

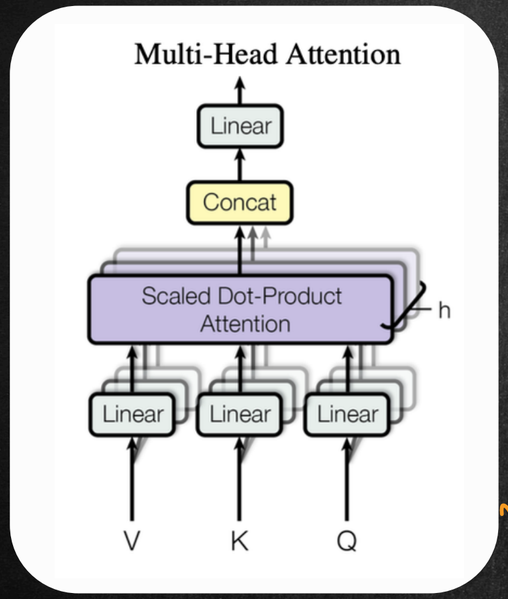
single head self-attention

i/p embedd → positional encodig → SDP → contextual embedd

APNA COLLEGE

Attention

Multi-Head Attention



multiple heads

"I like python." → i/p embed → positional modify → AH1 → CE1, AH2 → CE2, AH3 → CE3, ...

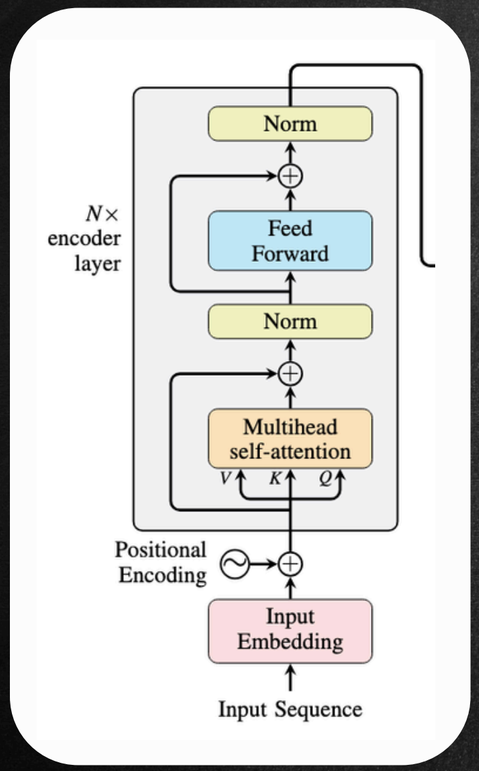
grammar

meaning

1. self-attention
2. cross-attention
3. masked attention

Encoder

Residual Connection



raw inp → directly → output connect

Add + Norm

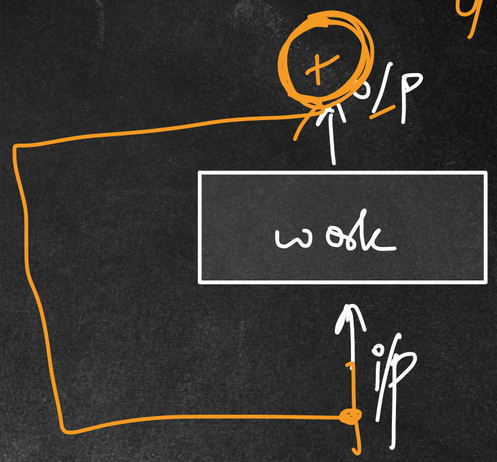
① vanishing gradient

② distorted info. in NN

residual connection

⇓

preserve original



Normalization

faster convergence

⇓

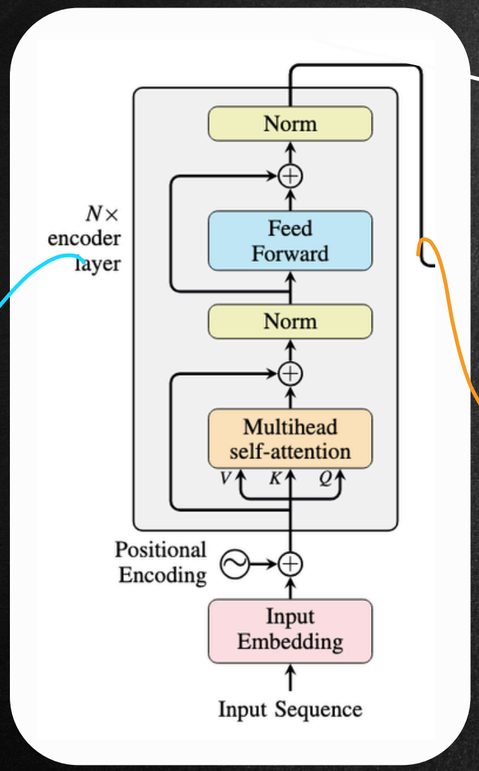
stabilize training

Encoder

FNN → adds non-linearity ⇓ " complexity → more enriched understanding

① gather infor.
② think deeply info.

→ imp features ↑↑
→ irrelevant info ↓↓



→ contextual embedd.

gpt2 → N = 48

gpt-3 → N = 96

APNA COLLEGE

Decoder

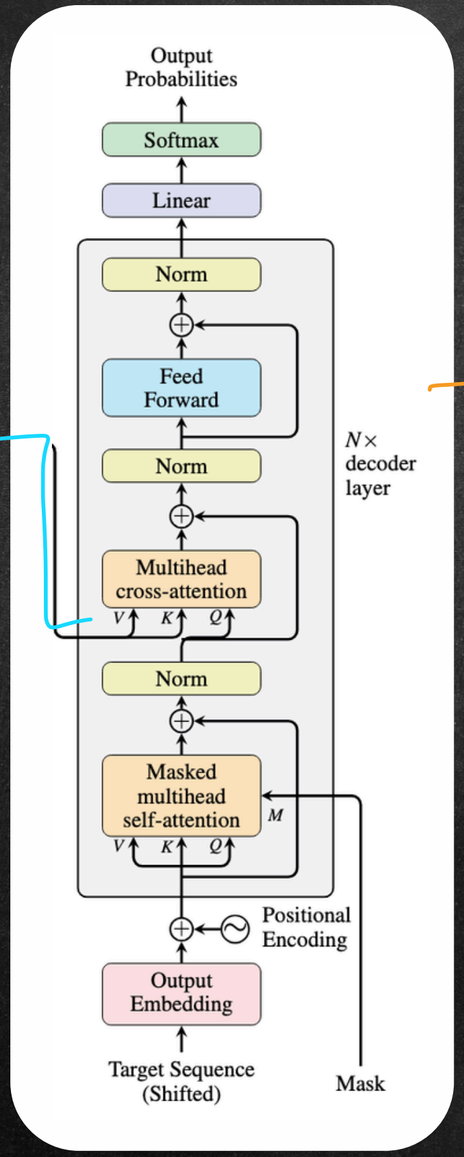
Cross-Attention

CROSS - ATTENTION

↓

attention btw 2 diff sequences

APNA COLLEGE



contextual embedding

I → ce1

like → ce2

Python → ce3

"I like Python"

<START>

मुझे

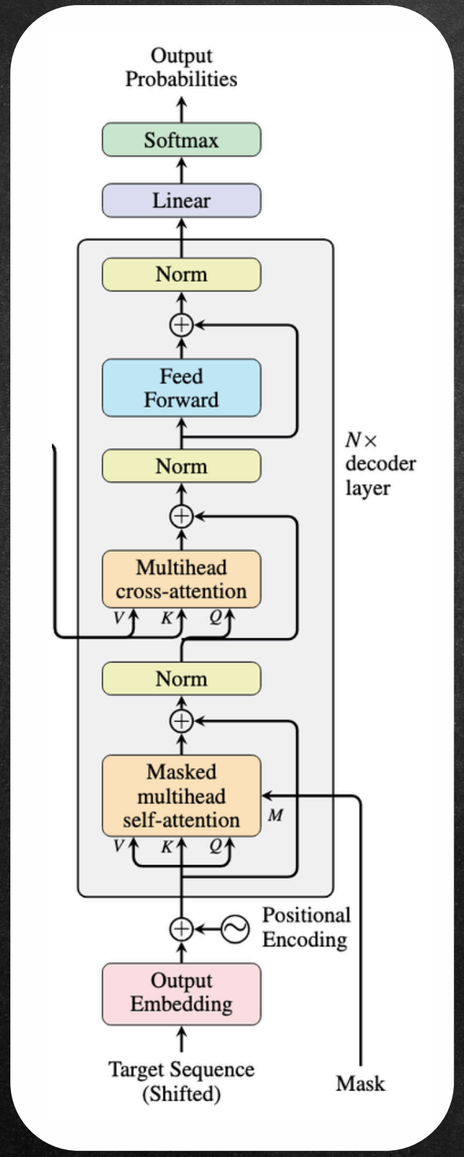
Q: next word?

पायथन → 80%

Decoder

Masked Attention

⇒ Apples and oranges are fruits.



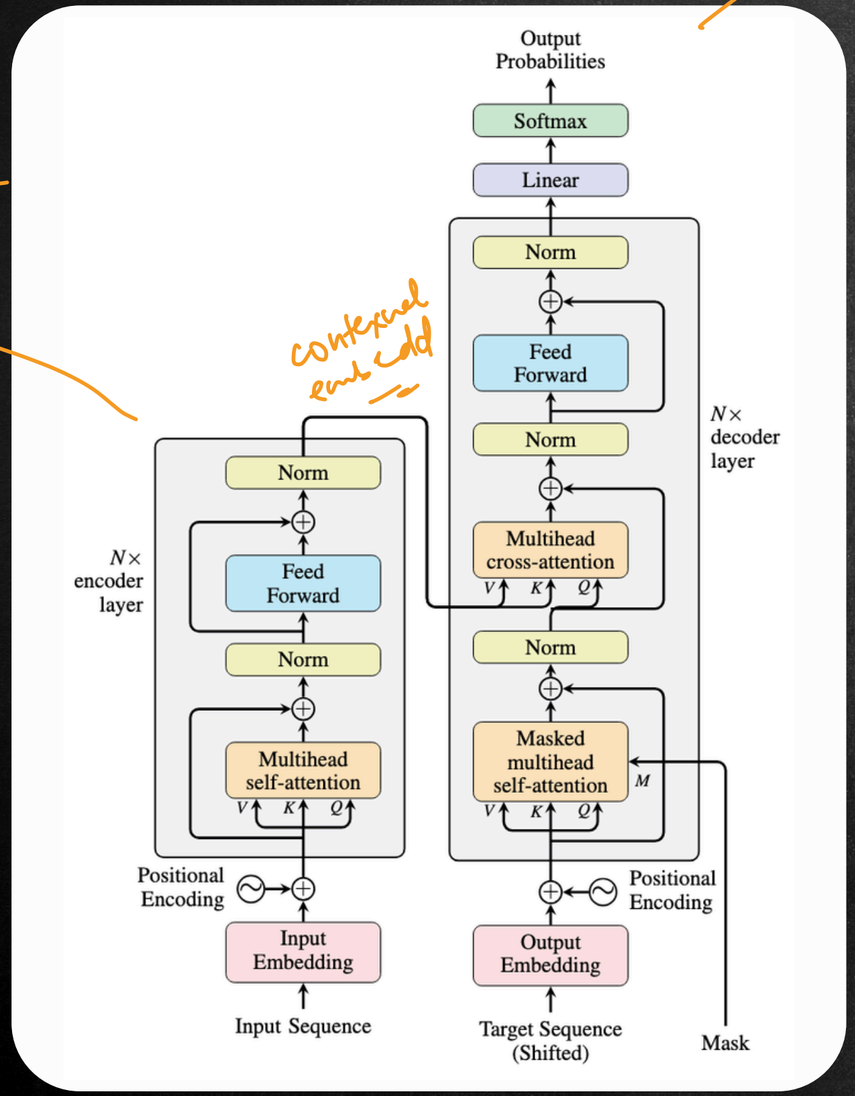
Training ⇒ { multi. head self-attention + mask

cheating ✗

APNA COLLEGE

Transformer

Architecture Summary



generation word by word

understand

contextual embedding

मुझे पढ़ना है — ?

prev o/p

Q: — पसंद

बुरा — 2%

पसंद — 72%

है — 10%

Pre-trained Transformers

BERT
GPT-3
T5
Text to Text Transfer Transformer

APNA COLLEGE

already trained on huge datasets for general tasks

⇓ fine tune

SOTA AI models

self-training

1. small dataset → poor generalization (overfitting) ⇒ better generalization
2. time ↓ → save
3. money ↓ → billions of params

Pre-trained Transformers

APNA COLLEGE

- Hugging Face
- PyTorch Hub
- TensorFlow Hub
- OpenAI APIs etc.

Generative Adversial Networks (GAN)

FNN
RNN
CNN
Transformer
GAN



- training unstable
- hard to evaluate

⇓

Diffusion models

i) rare dataset → training

ii) Ghibli trend

iii) Deep fakes

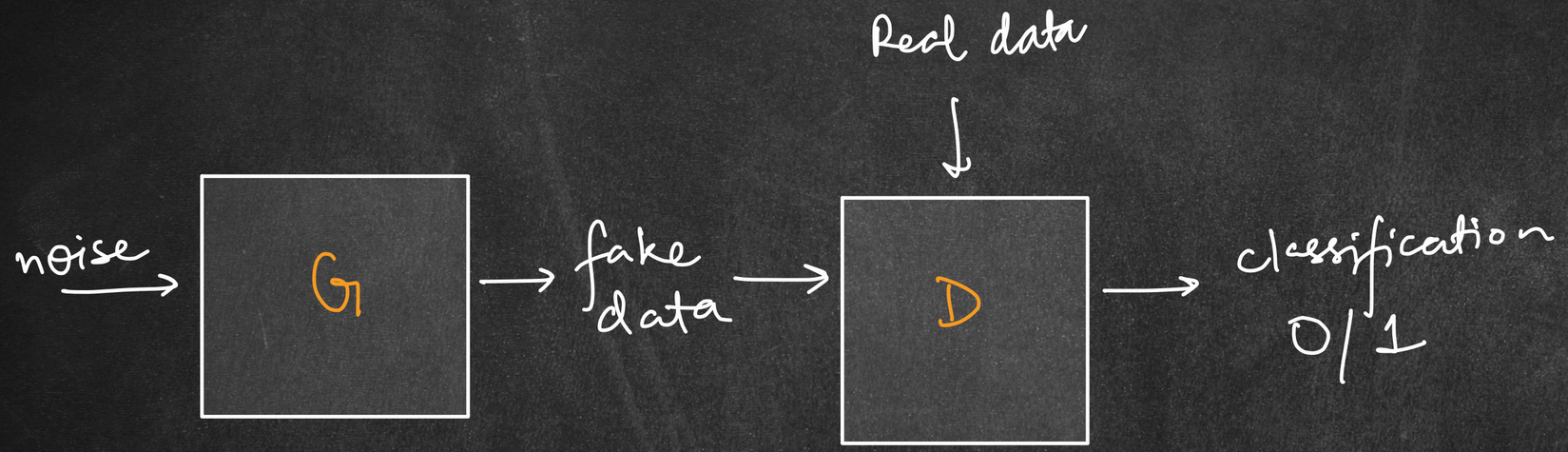
GANs

Architecture

2 NN

- Generator (G)
- Discriminator (D)

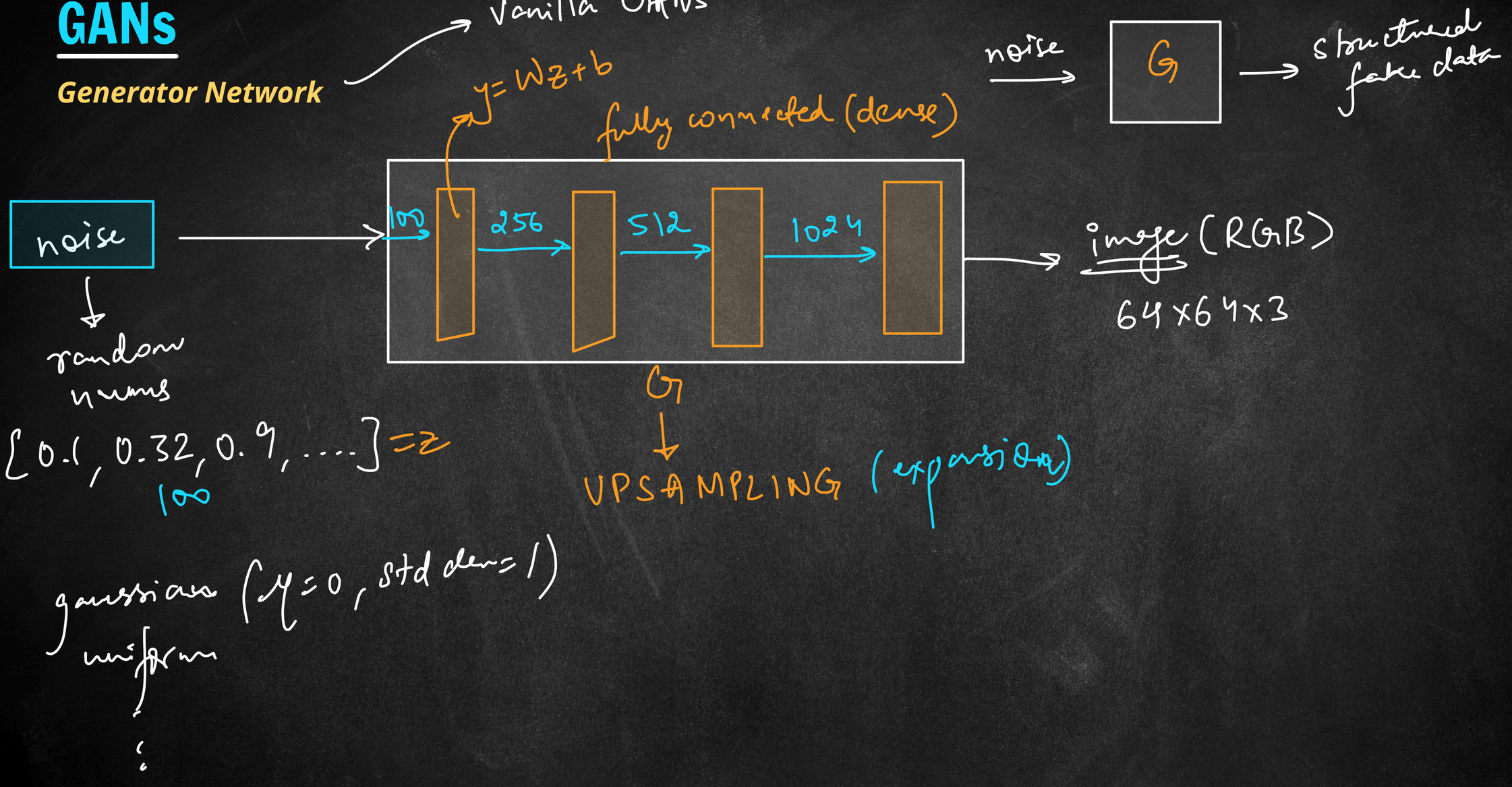
Generative Adversial Networks



GANs

Generator Network

Vanilla GANs



random nums

[0.1, 0.32, 0.9, ...] = z

100

gaussian ($\mu = 0$, std dev = 1)

uniform

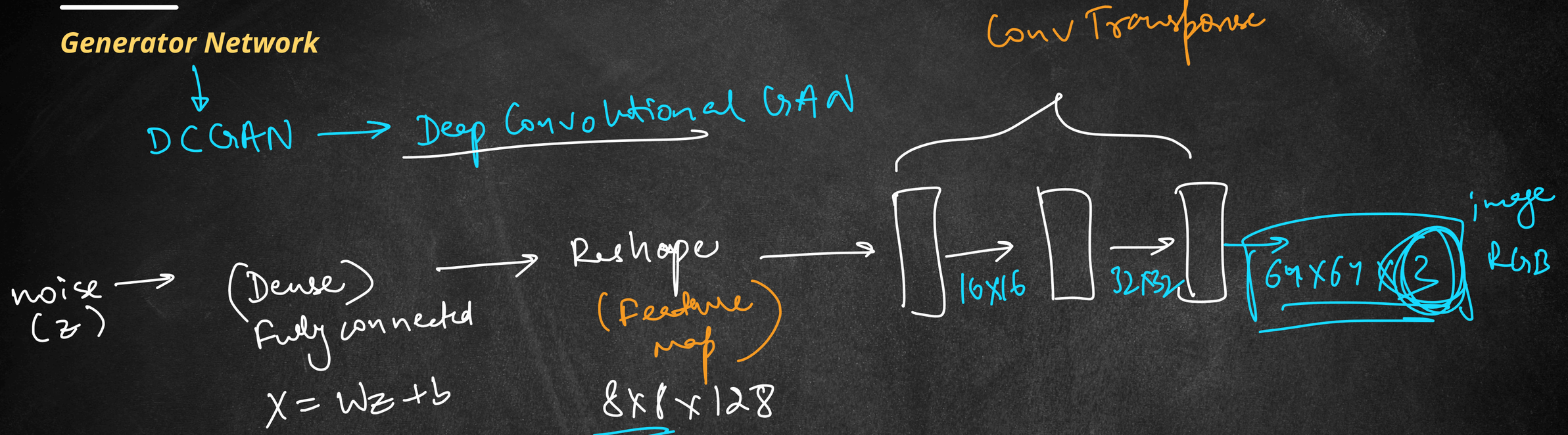
!

:

GANs

Generator Network

DCGAN → Deep Convolutional GAN



GANs



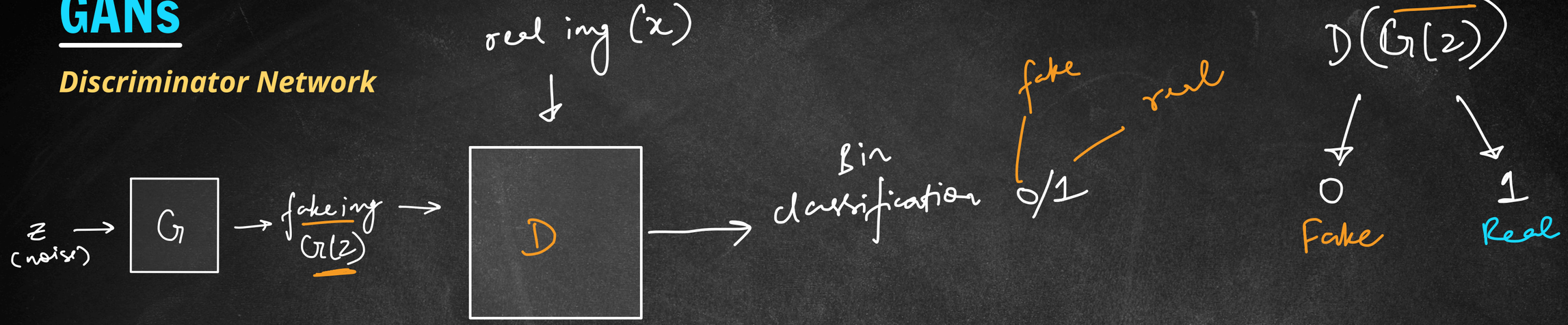
Generator Network - Code



APNA COLLEGE

GANs

Discriminator Network



(1) Generator Loss (minimize)

$$g_loss = -\log(D(G(z)))$$

(1) $\rightarrow \infty$

(2) $\rightarrow 0$ ✓

(2) Discriminator Loss

$$d_loss = -[\log(D(x)) + \log(1 - D(G(z)))]$$

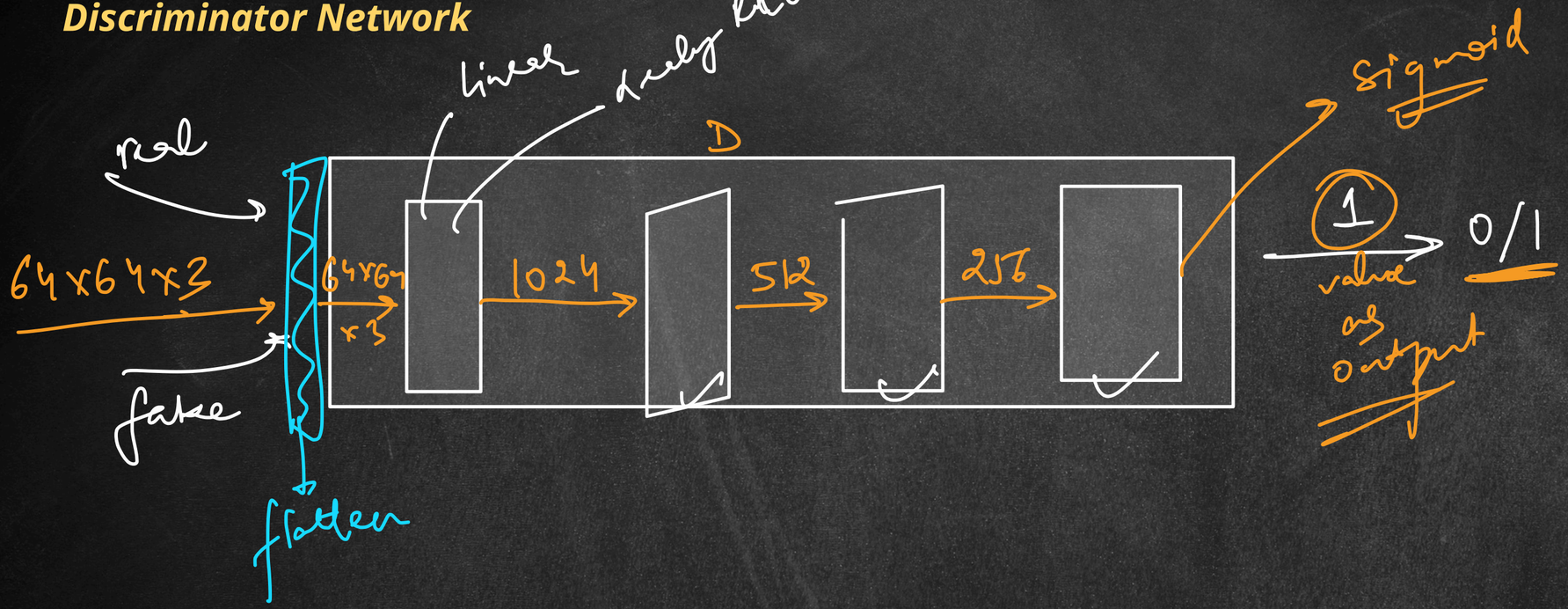
$$-[\log(1) + \log(1-0)] \rightarrow 0$$

GANs

→ vanilla GAN

CNN

Discriminator Network



Binary Cross Entropy

G → upsampling → generate

D → downsampling → classify

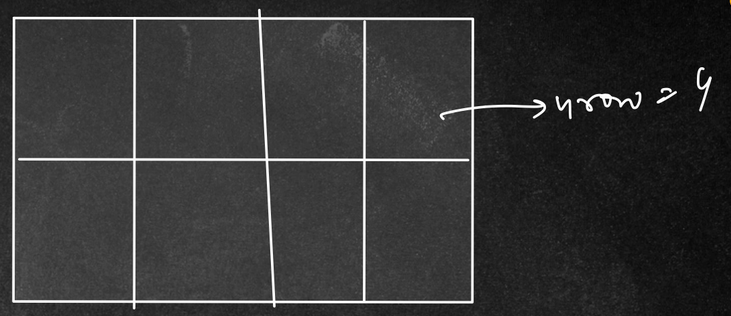
GANs

Discriminator Network - Code

APNA COLLEGE

GANs

Training



grid tensor ⟹ channels x H x W
0 1 2

matplotlib ⟹ H x W x C
1 2 0